

Final Project Report

Brain Tumor Segmentation using nnU-Net



Project Supervisor

Shafaq Nisar

Submitted By

F25PROJECT26450

Mutayyib Ali BC230416175

**Software Projects & Research Section,
Department of Computer Sciences,
Virtual University of Pakistan**

CERTIFICATE

This is to certify that **Mutayyib Ali (BC230416175)** has worked on and completed their Software Project at Software & Research Projects Section, Department of Computer Sciences, Virtual University of Pakistan in partial fulfillment of the requirement for the degree of BS in Computer Sciences under my guidance and supervision.

In our opinion, it is satisfactory and up to the mark and therefore fulfills the requirements of BS in Computer Sciences.

Supervisor / Internal Examiner

Shafaq Nisar

Supervisor,
Software Projects & Research Section,
Department of Computer Sciences
Virtual University of Pakistan

(Signature)

External Examiner/Subject Specialist

<<External Supervisor Name>>

(Signature)

Accepted By:

(For office use)

EXORDIUM

**In the name of Allah, the Compassionate, the
Merciful.**

**Praise be to Allah, Lord of Creation,
The Compassionate, the Merciful,
King of Judgment-day!**

**You alone we worship, and to You alone we pray
for help,
Guide us to the straight path**

The path of those who You have favored,

**Not of those who have incurred Your wrath,
Nor of those who have gone astray.**

DEDICATION

I dedicate this project to my beloved parents, whose unconditional love, endless patience, and continuous prayers have been my ultimate source of strength and inspiration throughout my academic journey. Their sacrifices have enabled me to reach this milestone.

I also dedicate this work to my esteemed teachers and mentors, whose guidance and imparted knowledge have illuminated my path and empowered me to tackle complex technological challenges.

ACKNOWLEDGEMENT

First and foremost, all praises and profound thanks are to Almighty Allah, the Compassionate and the Merciful, who bestowed upon me the strength, courage, and wisdom to complete this project successfully.

I would like to express my deepest and most sincere gratitude to my project supervisor, Shafaq Nisar, for her exceptional guidance, continuous encouragement, and invaluable feedback throughout the development of this project. Her expertise in deep learning and her patient mentorship has been instrumental in shaping this work and helping me navigate the complexities of neural network architecture.

Finally, I would like to extend my heartfelt thanks to my family and friends for their unwavering moral support, understanding, and motivation during the most challenging phases of this project's development.

PREFACE

The rapid advancement of Deep Learning in the field of medical image analysis has opened new frontiers for automated, highly accurate, and life-saving diagnostic tools. This project, titled "Brain Tumor Segmentation using nnU-Net," represents a comprehensive effort to leverage state-of-the-art 3D convolutional neural network architectures for a highly critical medical application: the precise identification and segmentation of brain tumors from multi-modal MRI scans.

This report documents the entire lifecycle of the project, developed in partial fulfillment of the requirements for the degree of BS in Computer Sciences at the Virtual University of Pakistan. It details the systematic approach taken to solve this complex problem, starting from the rigorous preprocessing of the BraTS 2021 dataset (handling FLAIR, T1, T1ce, and T2 modalities), to the implementation and training of the PyTorch-based nnU-Net model optimized with AdamW and Dice + Cross-Entropy loss functions. Furthermore, this document outlines the integration of the trained model into a fully functional Flask-based web application. This application was designed to bridge the gap between complex backend AI processing and accessible frontend clinical utility, allowing users to upload raw NIfTI MRI scans and visualize the predicted tumor segmentation masks in an intuitive interface.

The primary objective of this report is to provide a clear, structured, and technical walkthrough of the architectural design, development methodology, and post-processing techniques employed to achieve high-quality segmentation. It is my hope that this document serves not only as a reflection of the hard work invested in this academic endeavor but also as a meaningful demonstration of how artificial intelligence can be practically applied to assist in healthcare and medical diagnostics.

Table of Contents

CHAPTER 1: GATHERING & ANALYZING INFO	10
1.1 INTRODUCTION	10
1.2 PURPOSE	10
1.3 SCOPE	10
1.4 DEFINITIONS, ACRONYMS AND ABBREVIATIONS	12
1.5 PROJECT REQUIREMENTS	15
1.5.1 FUNCTIONAL REQUIREMENTS	15
1.5.2 NON-FUNCTIONAL REQUIREMENTS	17
1.6 USE CASES AND USAGE SCENARIOS.....	19
1.6.1 USE CASE DIAGRAM.....	19
1.6.2 USAGE SCENARIOS.....	20
1.7 DEVELOPMENT METHODOLOGY	23
1.7.1 CHOSEN METHODOLOGY	23
1.7.2 REASONS FOR CHOSEN METHODOLOGY	24
1.7.3 WORK PLAN (GANTT CHART)	24
1.7.4 PROJECT SCHEDULE (SUBMISSION CALENDAR)	26
CHAPTER 2: DESIGNING THE PROJECT	28
2.1 INTRODUCTION TO SYSTEM DESIGN	28
2.2 PROPOSED SYSTEM ARCHITECTURE OVERVIEW	28
2.3 DATASET SELECTION AND CHARACTERISTICS (BRATS 2021)	30
2.4 HANDLING MULTI-MODAL 3D MRI SCANS	31
2.5 DATA PREPROCESSING AND TRANSFORMATION PIPELINE	31
2.6 MEMORY MANAGEMENT STRATEGY: "LAZY LOADING"	31
2.7 INPUT SIZE CONFIGURATION AND CROPPING STRATEGY	32
2.8 THE 3D nnU-NET CONCEPTUAL ARCHITECTURE.....	32
2.9 POST-PROCESSING FRAMEWORK	32
2.10 INFERENCE METHODOLOGY: SLIDING WINDOW	32
2.11 WEB APPLICATION ARCHITECTURE (CLIENT-SERVER MODEL).....	33
2.12 BACKEND LOGIC AND PYTORCH INTEGRATION DESIGN.....	33
2.13 FRONTEND UI/UX DESIGN FOR MEDICAL VISUALIZATION	33
2.14 SUMMARY OF THE DESIGN PHASE	33
CHAPTER 3: DEVELOPMENT	35
3.1 DEVELOPMENT ENVIRONMENT SETUP.....	35
3.2 CORE LIBRARIES AND DEPENDENCIES	35
3.3 DATA STORAGE AND DIRECTORY MANAGEMENT	35
3.4 PARALLEL PROCESSING FOR DATA EXTRACTION	35
3.5 IMAGE PREPROCESSING PIPELINE	35
3.6 LABEL PREPROCESSING AND ENCODING.....	35
3.7 DYNAMIC DATA AUGMENTATION IMPLEMENTATION.....	36
3.8 PYTORCH DATASET AND DATALOADER INTEGRATION	36
3.9 CONVOLUTIONAL BLOCK (CONVBLOCK) IMPLEMENTATION	36

3.10 CONSTRUCTING THE NNUNET-ENCODER	36
3.11 CONSTRUCTING THE NNUNET-DECODER	36
3.12 LOSS FUNCTION AND OPTIMIZER INITIALIZATION	36
3.13 AUTOMATED HYPERPARAMETER TUNING	37
3.14 MIXED PRECISION TRAINING EXECUTION	37
3.15 EVALUATION METRICS CALCULATION LOGIC	37
3.16 MODEL CHECKPOINTING AND LOGGING	37
3.17 TEST-TIME AUGMENTATION (TTA) AND SLIDING WINDOW INFERENCE.....	37
3.18 FLASK WEB APPLICATION BACKEND SETUP	37
3.19 INTEGRATING PYTORCH INFERENCE WITH FLASK	38
3.20 FRONTEND IMPLEMENTATION AND VISUALIZATION RENDERING	38
 CHAPTER 4: RESULTS	40
 4.1 QUANTITATIVE ANALYSIS OF MODEL METRICS	40
4.1.1 TRAINING AND VALIDATION CONVERGENCE	40
4.1.2 OVERLAP METRICS: DICE AND IOU	40
4.1.3 DIAGNOSTIC ACCURACY: SENSITIVITY AND SPECIFICITY.....	40
4.2 QUALITATIVE VISUAL RESULTS AND COMPARATIVE ANALYSIS	42
4.2.1 MULTI-MODAL VISUALIZATION AND PREDICTION	42
4.2.2 IMPACT OF POST-PROCESSING REFINEMENT	42
4.2.3 SUMMARY OF FINDINGS	44
 CHAPTER 5: USER INTERFACE.....	46
 5.1 DESIGN PHILOSOPHY AND INTERFACE ARCHITECTURE	46
5.2 STEP 1: MULTI-MODAL DATA INGESTION AND VALIDATION	46
5.3 STEP 2: SEGMENTATION EXECUTION AND MONITORING.....	47
5.4 STEP 3: RESULT VISUALIZATION AND SUB-REGION ANALYSIS.....	48

CHAPTER 1

Gathering & Analyzing Info

CHAPTER 1: Gathering & Analyzing Info

1.1 INTRODUCTION

Brain tumors are a very serious and dangerous medical condition. To treat a patient properly, doctors need to know exactly where the tumor is, how big it is, and what shape it has. To look inside a patient's brain without surgery, doctors use a machine called an MRI (Magnetic Resonance Imaging) scanner. This machine takes detailed 3D pictures of the brain. However, finding the exact edges of the tumor in these 3D pictures is a very slow and difficult job. A doctor has to look at hundreds of image slices one by one and manually draw a line around the tumor. This takes a lot of time and can be exhausting.

This project, "Brain Tumor Segmentation using nnU-Net," uses modern computers and Artificial Intelligence (AI) to solve this problem. We are using a very smart computer program called a "deep learning model" to look at the MRI pictures and automatically find the tumor. The word "segmentation" simply means dividing an image into parts—in our case, separating the tumor from the healthy brain tissue.

For this project, we are using a highly advanced AI model called "nnU-Net". We are teaching this model how to recognize tumors by showing it thousands of examples from a famous dataset called BraTS 2021. Because we want this tool to be very easy for people to use, we are also building a web application using a tool called Flask.

Instead of writing complex computer code, a user or a doctor can simply open a website, upload their MRI files, and click a button. The website will then use our trained AI model to find the tumor and show clear results on the screen.

1.2 PURPOSE

The main purpose of this project is to build an automated and highly accurate system that can find brain tumors in medical images without human help.

Currently, manual segmentation (drawing the tumor by hand) takes too much of a doctor's valuable time. When doctors are tired, they might also make small mistakes. By creating an artificial intelligence system, we want to speed up this process. The computer can look at a 3D brain scan and find the exact location and size of the tumor in just a few seconds or minutes. This fast process will help medical experts save time so they can focus on planning the best treatment for the patient.

Another very important purpose of this project is to build a simple and friendly website (User Interface). AI models are usually very complex and require advanced computer skills to use. We want to resolve this difficulty. By building a Flask web application, we are creating a bridge between the complex AI brain and the normal user. The purpose is to allow anyone to upload their multi-modal MRI scans safely and easily view the original brain image side-by-side with the AI-predicted tumor mask.

Ultimately, the purpose is to show how artificial intelligence can be practically and easily used to assist in modern healthcare.

1.3 SCOPE

The project covers the development of an intelligent, deep learning-based system capable of performing automated brain tumor segmentation on MRI images using the nnU-Net framework, along with a user-friendly web interface for model interaction. The system will include all essential functionalities required for processing MRI scans,

running inference through a trained nnU-Net model, and visualizing segmentation outputs. It will allow users to upload MRI slices or volumes, automatically preprocess the input, generate predicted tumor masks, and display the segmentation results alongside ground-truth masks for comparison. The software will also provide essential feedback to users regarding model confidence, processing status, and error handling, ensuring smooth interaction. Additionally, the scope includes training, validating, and evaluating the nnU-Net model on the BraTS 2021 dataset, optimizing it for accuracy, robustness, and clinically meaningful segmentation performance. The web application will be built using Flask and will integrate backend inference pipelines with a simple, intuitive frontend visualization dashboard.

The main objective of the project is to design and implement a reliable, accurate, and user-accessible brain tumor segmentation system that demonstrates the practical application of deep learning in medical imaging. The project aims to automate the segmentation process, improve the efficiency of tumor identification, and provide a platform that visually communicates model performance to end-users, such as researchers or clinicians. Additional objectives include achieving robust model performance across tumor sub-regions, ensuring the web application is intuitive and responsive, and validating the system through standard evaluation metrics like Dice score. The project also aims to highlight the potential of nnU-Net as a self-configuring architecture capable of delivering state-of-the-art results with minimal manual intervention.

The deliverables of the project will include a fully trained nnU-Net segmentation model, a documented preprocessing and training pipeline, evaluation reports detailing model accuracy and performance metrics, and a complete Flask-based web application enabling MRI upload, inference, and visualization of segmentation results. Additional deliverables will include the source code repository, system documentation, an SRS document, deployment instructions, and a final project report summarizing methodology, results, and conclusions. Together, these deliverables will demonstrate the technical implementation and practical usability of an automated brain tumor segmentation system.

The project will operate under practical constraints such as limited computational resources for model training, requiring reliance on GPU availability and manageable training times. Time constraints include adherence to academic project deadlines, influencing the scope of model optimization and the extent of hyperparameter tuning possible. Technological constraints include using the nnU-Net framework in its recommended configuration, Flask for web deployment, and standard Python libraries for preprocessing and visualization. The accuracy of the segmentation model will be limited by the quality and variability of the BraTS dataset, the chosen model configuration, and available hardware.

The project will operate under practical constraints such as limited computational resources for model training, requiring reliance on GPU availability and manageable training times. Time constraints include adherence to academic project deadlines, influencing the scope of model optimization and the extent of hyperparameter tuning possible. Technological constraints include using the nnU-Net framework in its recommended configuration, Flask for web deployment, and standard Python libraries for preprocessing and visualization. The accuracy of the segmentation model will be limited by the quality and variability of the BraTS dataset, the chosen model configuration, and available hardware.

1.4 DEFINITIONS, ACRONYMS AND ABBREVIATIONS

To make this report easy to read, here is a list of important technical words and what they mean in simple English:

1. AdamW: An advanced optimization algorithm used during the training of the neural network to help the model learn faster and more accurately.
2. AI (Artificial Intelligence): The simulation of human intelligence by computers, enabling them to perform tasks like recognizing images and making decisions.
3. API (Application Programming Interface): A software bridge that allows the frontend (the website) to communicate with the backend (the AI model).
4. Backend: The "brain" or hidden part of the web application where data processing, file saving, and AI predictions happen.
5. Batch Size: The number of MRI scans the AI looks at simultaneously before updating its memory during training.
6. Bottleneck: The deepest and most compressed middle section of the U-Net architecture where the most complex patterns are stored.
7. BraTS (Brain Tumor Segmentation): A massive, globally recognized dataset of brain MRI scans used by researchers to train and test medical AI models.
8. CE (Cross-Entropy) Loss: A mathematical formula used during training to penalize the AI when it misclassifies a background pixel as a tumor pixel.
9. CNN (Convolutional Neural Network): A specific type of artificial intelligence designed specifically to look at, analyze, and understand images.
10. Convolution: A mathematical operation where a small virtual filter slides over an image to detect shapes, edges, and textures.
11. CRF (Conditional Random Field): A mathematical post-processing tool used to clean up the blurry edges of an AI prediction, making the tumor boundaries sharp and accurate.
12. CSS (Cascading Style Sheets): A programming language used to style the web application, adding colors, layouts, and fonts to the UI.
13. Data Augmentation: A technique used to artificially create more training data by slightly rotating, zooming, or adding noise to the original MRI scans.
14. Decoder: The second half of the U-Net model that takes compressed information and expands it back into a full-sized 3D segmentation mask.
15. Dice Loss / Dice Score: A critical mathematical metric that measures the percentage of overlap between the AI's predicted tumor and the human doctor's actual drawn tumor.
16. DL (Deep Learning): A highly advanced subset of machine learning that uses multi-layered artificial neural networks.

17. Down sampling (Pooling): Reducing the physical dimensions of an image in the network to save computer memory and focus on the "big picture" shapes.
18. Edema: The swelling or accumulation of fluid in the healthy brain tissue immediately surrounding the main brain tumor.
19. Encoder: The first half of the U-Net model that takes a large 3D image and shrinks it down to extract core features and patterns.
20. Epoch: One complete cycle where the AI model has looked at the entire dataset once during the training phase.
21. ET (Enhancing Tumor): The highly active, aggressive part of the tumor core that lights up brightly on a T1ce MRI scan.
22. FLAIR (Fluid-Attenuated Inversion Recovery): An MRI modality that darkens normal brain fluids, making it very easy to see swelling (edema) around the tumor.
23. Flask: A lightweight and simple web framework written in Python, used to build the website interface for this project.
24. Frontend: The visible, interactive part of the web application that users see and click on in their internet browser.
25. Ground Truth: The 100% correct, historically accurate answer. In this project, it refers to the tumor masks carefully drawn by expert human radiologists.
26. GUI (Graphical User Interface): The visual screens, buttons, and layouts that allow users to interact with the software easily without writing code.
27. HTML (Hypertext Markup Language): The standard coding language used to create the basic structure and elements of web pages.
28. Hyperparameter: A manual setting or dial (like learning rate or batch size) chosen by the programmer before training begins to control how the AI learns.
29. Inference: The testing phase where a fully trained AI model is given a brand-new MRI scan, it has never been seen before, to predict where the tumor is.
30. Instance Normalization: A mathematical trick applied inside the neural network to keep numerical values stable, making the AI easier to train on 3D medical images.
31. IoU (Intersection over Union): A strict evaluation metric that calculates the area of overlap divided by the total area of both the prediction and the ground truth.
32. LeakyReLU: An activation function in the neural network that helps the computer learn complex patterns by allowing small negative numbers to pass through instead of blocking them.
33. Loss Function: A scoring system that calculates exactly how "wrong" the AI model's current guess is. The AI's goal is to minimize this score.

34. ML (Machine Learning): A field of computer science where computers are taught to learn from data without being explicitly programmed for every scenario.
35. Modality: A specific type of setting or filter used during an MRI scan (e.g., T1, T2, FLAIR) to highlight different biological tissues.
36. Morphological Operations: Post-processing techniques (like "Erosion" and "Dilation") used to automatically erase tiny false speckles of noise from the final AI prediction.
37. Necrosis: Dead tissue found inside the core of aggressive brain tumors.
38. NIFTI (.nii / .nii.gz): The standard global file format used to store highly detailed 3D medical imaging data.
39. nnU-Net: Stands for "no-new-Net." An extremely robust, state-of-the-art framework that automatically configures U-Net architectures for medical image segmentation.
40. NumPy: A popular Python library used to handle complex mathematics and large arrays of numbers.
41. Pixel: A 2D picture element; a single tiny dot on a flat computer screen.
42. PyTorch: A powerful and widely used software library created by AI researchers to easily build, train, and run deep neural networks.
43. SciPy: A Python library used for scientific and technical computing, utilized in this project for cleaning up the final mask images.
44. Segmentation: The act of drawing strict boundaries around a specific object in an image. Here, it means isolating the tumor from the healthy brain.
45. Skip Connection: A special "shortcut" bridge inside a U-Net that passes high-resolution spatial details directly from the encoder to the decoder, preventing blurry outputs.
46. T1 / T1-weighted: A standard MRI modality that clearly shows the normal anatomy and structure of the brain.
47. T1ce (T1-contrast enhanced): An MRI modality where the patient is injected with a special dye to make the active tumor core glow brightly.
48. T2 / T2-weighted: An MRI modality that highlights watery tissues, making it useful for seeing the spread of the tumor.
49. TC (Tumor Core): The main, dense body of the tumor (including enhancing and dead tissue), ignoring the outer swelling.
50. Training: The highly intensive computational phase where the AI repeatedly reviews thousands of images to learn the patterns of a brain tumor.

51. U-Net: A highly famous type of Convolutional Neural Network shaped like the letter "U", originally invented specifically for biomedical image segmentation.
52. Up sampling: The process inside the decoder artificially increases the size of a compressed image back to its original 3D dimensions.
53. Validation: A mini-test run at the end of every training epoch to check if the AI is genuinely learning or just memorizing the data.
54. Voxel: A "Volumetric Pixel"; the 3D equivalent of a pixel representing a tiny cube of tissue inside an MRI scan.
55. WT (Whole Tumor): The complete extent of the disease, including the core, the enhancing regions, and all surrounding edema/swelling.

1.5 PROJECT REQUIREMENTS

1.5.1 Functional Requirements

Following are the functional requirements of the project:

Dataset Handling & Preprocessing Requirements

- The system shall load and preprocess the BraTS 2021 dataset, including all MRI modalities (T1, T1CE, T2, FLAIR).
- The system shall perform intensity normalization, resampling, cropping, and spacing adjustments following nnU-Net preprocessing standards.
- The system shall apply data augmentation techniques such as flipping, rotation, scaling, elastic deformation, and noise injection.
- The system shall convert segmentation labels into the required one-hot encoded representations (WT, TC, ET).
- The system shall provide visualization utilities for displaying MRI slices and their ground truth masks.
- The system shall ensure consistent preprocessing for both training and inference pipelines.

Model Development & Training Requirements

- The system shall implement the nnU-Net architecture using PyTorch, following its automatic configuration rules.
- The system shall implement a complete training pipeline, including patch extraction, batch creation, forward pass, loss calculation, and backpropagation.
- The system shall use Dice Loss combined with Cross-Entropy Loss for segmentation training.
- The system shall use the AdamW optimizer with appropriate learning rate scheduling.
- The system shall track key training metrics, including loss curves, Dice Score, IoU, and class-wise accuracy.
- The system shall store model checkpoints at defined intervals to allow training resumption or rollback.
- The system should implement runtime monitoring such as progress indicators and epoch summaries.

Model Evaluation & Visualization Requirements

- The system evaluates the model using metrics including Dice Score, IoU, Precision, and Recall.
- The system shall generate visual comparisons of predicted segmentation masks versus ground truth masks.
- The system shall provide a side-by-side comparison of nnU-Net and U-Net model predictions.
- The system shall produce evaluation reports containing performance graphs, confusion matrices, and segmentation examples.
- The system shall store evaluation results and visualizations for documentation and final reporting.

Flask Web Application Requirements

- The system should provide a Flask-based web interface allowing users to upload MRI scans.
- The system shall support uploads in common image formats.
- The system shall preprocess the files to match the model's expected input format.
- The system shall generate segmentation predictions using the trained nnU-Net model.
- The system shall display the following to the user:
 - Original MRI slice
 - Predicted segmentation mask
 - Ground truth mask (optional upload)
- The system shall show meaningful error messages for incorrect formats, corrupt files, or missing modalities.
- The system shall log upload events, processing durations, and predict outcomes.
- The system shall maintain a clean, intuitive, and responsive user interface.

Post-Processing & Optimization Requirements

- The system shall support optional post-processing techniques such as CRF, connected component filtering, or morphological operations.
- The system shall provide hyperparameter tuning options (learning rate, batch size, epochs, patch size).
- The system should monitor validation metrics and stop training early if performance plateaus (optional).
- The system shall allow experimentation with different nnU-Net configurations for performance improvement.

File Management & Logging Requirements

- The system shall organize dataset files, model checkpoints, logs, and outputs into a structured directory format.
- The system shall store detailed logs of training, evaluation, and inference operations.
- The system shall maintain reproducibility by saving configuration files, hyperparameters, and environment details.

Documentation Requirements

- The system shall provide documentation for dataset preparation, preprocessing, training, evaluation, and deployment steps.
- The system shall include instructions for installing dependencies, setting up the Python environment, and executing the web application.

The system should include model architecture diagrams, workflow pipelines, and usage guides.

1.5.2 Non-Functional Requirements

Following are the non-functional requirements of the project:

Performance Requirements

- The system shall process an uploaded MRI scan and generate a segmentation prediction within an acceptable response time.
- The system shall load the trained nnU-Net model into memory efficiently to minimize initialization delays.
- The system shall ensure that training throughput remains stable, utilizing GPU acceleration where available to achieve optimal training speed.
- The system shall maintain real-time responsiveness in the Flask web interface, ensuring that UI interactions and status updates appear without noticeable lag.
- The system shall support handling high-resolution MRI images without system crashes or excessive memory usage.
- The system shall ensure that memory consumption remains within hardware limits by using mini-batch training and patch-based processing.

Reliability Requirements

- The system shall ensure consistent execution of preprocessing, prediction, and visualization pipelines without unexpected failures.
- The system shall automatically create backup model checkpoints to prevent data loss in case of training interruptions.
- The system shall generate meaningful error logs for debugging, recording the cause and context of failures.
- The system should maintain consistent segmentation performance across multiple runs by ensuring deterministic preprocessing and fixed random seeds (where possible).
- The system shall include validation checks that prevent corrupted, incomplete, or incompatible MRI files from being processed incorrectly.

Availability Requirements

- The Flask web application shall remain available for users during normal usage hours without requiring frequent restarts.
- The system shall automatically restart inference services when critical errors occur, ensuring minimal downtime.
- The local or server environment hosting the web application shall maintain at least 95% availability during demonstration or testing periods.

Security Requirements

- The system should validate user-uploaded files to prevent malicious inputs, corrupted files, or unsupported formats.
- The system shall ensure that user-uploaded MRI data is not stored permanently unless explicitly needed for analysis.

- The system shall prevent unauthorized modifications to model checkpoints, source code, or training logs.

Usability Requirements

- The user interface shall be simple, clean, and intuitive for users with minimal technical background, such as medical personnel or students.
- The UI shall display segmentation results in a visually distinguishable format (e.g., distinct colors for tumor regions) for easy understanding.
- The system shall guide users with clear instructions for uploading MRI scans, interpreting results, and understanding segmentation overlays.
- The system should provide meaningful error messages when users upload invalid files or missing modalities.
- The web application shall support standard desktop browsers such as Chrome, Edge, and Firefox without requiring plugins.
- The system should minimize user input complexity, allowing segmentation with a few clicks.

Maintainability Requirements

- The system shall follow modular programming practices, allowing us to update preprocessing, model training, or web components independently.
- The source code shall include comments, documentation, and clear folder structure for easy maintenance.
- The system shall use configuration files (YAML/JSON) for model settings to simplify future modifications.
- The system should provide separate modules for dataset handling, model creation, training, evaluation, and web deployment.
- The system shall ensure that updating libraries (PyTorch, Flask, etc.) does not break core functionality by using version-locked environments (requirements.txt).

Scalability Requirements

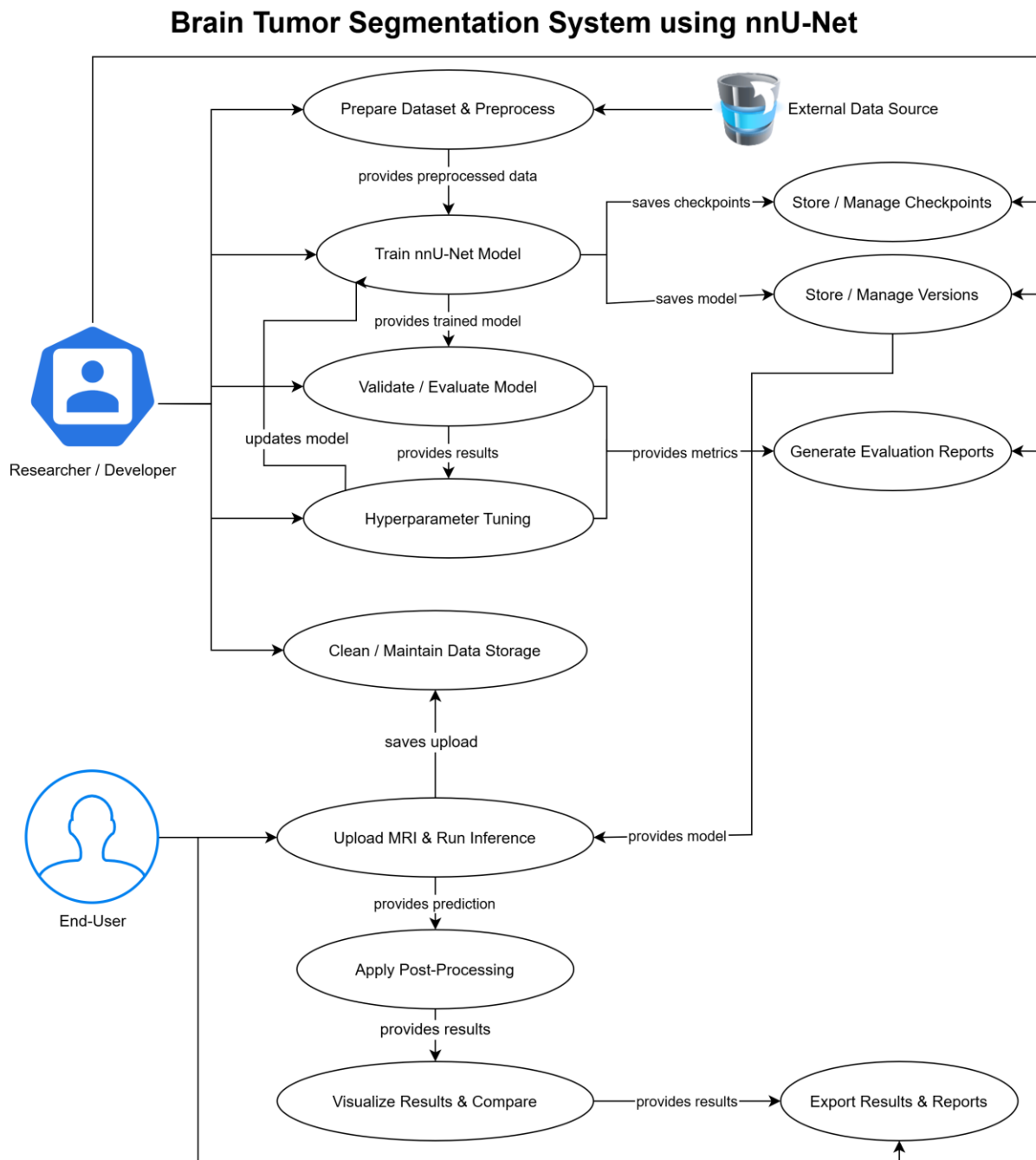
- The system shall support moving from local execution to server/cloud deployment without major code modifications.
- Architecture shall allow retraining the model with larger datasets or new MRI modalities in the future.
- The web application shall support multiple users performing inference sequentially without performance degradation.

Portability Requirements

- The system should run on Windows, Linux, or macOS environments where Python, PyTorch, and Flask are supported.
- The trained model shall be portable, allowing it to be executed on different hardware (CPU/GPU) without retraining.
- The system shall be deployable on local machines, university servers, or cloud platforms with minimal configuration changes.

1.6 USE CASES AND USAGE SCENARIOS

1.6.1 Use Case Diagram



1.6.2 Usage Scenarios

Following are the usage scenarios for the use cases in the above use case diagram.

Scenario 1

FIELD	DESCRIPTION
TITLE	Prepare Dataset & Preprocess
ACTORS	Researcher / Developer, External Data Source
DESCRIPTION	The researcher loads MRI scans and applies preprocessing steps such as normalization, resampling, cropping, augmentation, and formatting according to nnU-Net standards.
PRE-CONDITIONS	Dataset available; storage accessible; system installed with preprocessing modules.
TASK SEQUENCE	1. Researcher selects raw dataset.2. System loads modalities.3. Preprocessing pipeline runs.4. Preprocessed data saved.
EXCEPTIONS	Missing/corrupted MRI files; unsupported formats; insufficient storage.
POST-CONDITIONS	A fully preprocessed dataset is stored and ready for training.
AUTHOR	Mutayyib Ali

Scenario 2

FIELD	DESCRIPTION
TITLE	Train nnU-Net Model
ACTORS	Researcher / Developer
DESCRIPTION	The system trains the nnU-Net model using preprocessed data while saving checkpoints and model versions.
PRE-CONDITIONS	Preprocessed dataset available; training configuration defined.
TASK SEQUENCE	1. Researcher initiates training. 2. System loads data and starts with epochs. 3. Loss computed, and backpropagation performed. 4. Checkpoints saved. 5. Final trained model stored.
EXCEPTIONS	GPU memory errors; training crashes; invalid data patches.
POST-CONDITIONS	Trained models and checkpoints are saved for evaluation.
AUTHOR	Mutayyib Ali

Scenario 3

FIELD	DESCRIPTION
TITLE	Validate & Evaluate Model
ACTORS	Researcher / Developer
DESCRIPTION	The trained model is evaluated using validation data to generate metrics and performance graphs.
PRE-CONDITIONS	Trained model exists; validation dataset prepared.
TASK SEQUENCE	1. Researcher starts evaluation. 2. System loads validation set. 3. Predictions generated.

	4. Metrics computed. 5. Report generated.
EXCEPTIONS	Missing validation data; metric computation errors.
POST-CONDITIONS	Evaluation report and performance metrics stored.
AUTHOR	Mutayyib Ali

Scenario 4

FIELD	DESCRIPTION
TITLE	Hyperparameter Tuning
ACTORS	Researcher / Developer
DESCRIPTION	Researchers modify training settings to improve accuracy, and the system retrains accordingly.
PRE-CONDITIONS	Initial model performance results available.
TASK SEQUENCE	1. Researcher selects parameters to tune. 2. System retrains model. 3. Metrics re-evaluated. 4. Results compared with earlier versions.
EXCEPTIONS	Unstable settings cause training failure; incompatible parameters.
POST-CONDITIONS	Optimized model configuration saved.
AUTHOR	Mutayyib Ali

Scenario 5

FIELD	DESCRIPTION
TITLE	Store & Manage Checkpoints
ACTORS	System (Automatic), Researcher
DESCRIPTION	System automatically saves intermediate checkpoints and maintains versioned models.
PRE-CONDITIONS	Training process initiated.
TASK SEQUENCE	1. System generates checkpoints. 2. Saves them in structured folders. 3. Researcher optionally restores or deletes checkpoints.
EXCEPTIONS	Disk full; write-permission errors.
POST-CONDITIONS	Securely stored checkpoints are available for rollback.
AUTHOR	Mutayyib Ali

Scenario 6

FIELD	DESCRIPTION
TITLE	Generate Evaluation Reports
ACTORS	Researcher / Developer
DESCRIPTION	System compiles segmentation outputs, metrics, and graphs into an evaluation report.
PRE-CONDITIONS	Model evaluation completed; metrics computed.
TASK SEQUENCE	1. Researcher requests report. 2. System compiles metric tables & images. 3. Report exported to PDF/HTML.

EXCEPTIONS	Missing result data; report creation errors.
POST-CONDITIONS	Evaluation report available for documentation or presentation.
AUTHOR	Mutayyib Ali

Scenario 7

FIELD	DESCRIPTION
TITLE	Upload MRI & Run Inference
ACTORS	End-User
DESCRIPTION	User uploads an MRI scan through the Flask web interface, and the model generates segmentation.
PRE-CONDITIONS	Web app running; user has MRI file in supported format.
TASK SEQUENCE	1. User uploads MRI file. 2. System preprocesses input. 3. Model performs inference. 4. Prediction mask generated.
EXCEPTIONS	Unsupported file format; corrupt file; model loading failure.
POST-CONDITIONS	Prediction stored for visualization and export.
AUTHOR	Mutayyib Ali

Scenario 8

FIELD	DESCRIPTION
TITLE	Apply Post-Processing
ACTORS	System, End-User
DESCRIPTION	System refines segmentation using CRF, morphological filtering, or connected components.
PRE-CONDITIONS	Raw prediction mask available.
TASK SEQUENCE	1. System receives prediction. 2. Applies selected post-processing method. 3. Generates cleaned mask.
EXCEPTIONS	Parameter errors; CRF failure; post-processing mismatch.
POST-CONDITIONS	Improved segmentation mask ready for visualization.
AUTHOR	Mutayyib Ali

Scenario 9

FIELD	DESCRIPTION
TITLE	Visualize Results & Compare
ACTORS	End-User
DESCRIPTION	System displays original MRI, predicted mask, and optional ground truth for side-by-side comparison.
PRE-CONDITIONS	Prediction and post-processing completed.
TASK SEQUENCE	1. User requests visualization. 2. Original MRI displayed. 3. Predicted mask overlaid. 4. Ground truth shown if available.
EXCEPTIONS	Rendering issues; invalid ground truth format.

POST-CONDITIONS	User views segmentation quality visually.
AUTHOR	Mutayyib Ali

Scenario 10

FIELD	DESCRIPTION
TITLE	Export Results & Reports
ACTORS	End-User
DESCRIPTION	User exports segmentation masks, graphs, or evaluation reports.
PRE-CONDITIONS	Predictions, visualizations, or evaluation data available.
TASK SEQUENCE	1. User selects export. 2. System generates downloadable files. 3. User downloads results.
EXCEPTIONS	File generation errors; missing data.
POST-CONDITIONS	Users obtain exported files for documentation or clinical use.
AUTHOR	Mutayyib Ali

Scenario 11

FIELD	DESCRIPTION
TITLE	Clean & Maintain Data Storage
ACTORS	System, Researcher
DESCRIPTION	System removes outdated uploads, logs, and temporary files to manage storage.
PRE-CONDITIONS	Storage contains unnecessary or outdated files.
TASK SEQUENCE	1. System scans storage. 2. Researcher chooses cleanup options (optional). 3. System deletes selected files.
EXCEPTIONS	Permission errors; accidental selection of in-use files.
POST-CONDITIONS	Storage cleaned and optimized for continued use.
AUTHOR	Mutayyib Ali

1.7 DEVELOPMENT METHODOLOGY

1.7.1 Chosen Methodology

For this project, I have chosen the Agile Methodology (specifically an Iterative Development approach).

Unlike older methods (like the Waterfall method) where one has to perfectly finish one step before moving to the next, the Agile methodology allows me to build the project in small, repeated cycles called "iterations." In an Artificial Intelligence and Machine Learning project like this one, it is almost impossible to get everything perfect on the first try. I must build a small part, test it, learn from my mistakes, and improve it.

Using the Agile methodology, I have divided this project into four main, manageable phases:

- **Phase 1: Data Preparation (The Foundation):** In this first step, I focus only on the BraTS 2021 dataset. I load the raw 3D MRI scans (FLAIR, T1, T1ce, and T2), clean them, handle missing values, and convert them into the correct formats (.nii.gz) that my AI model can understand.
- **Phase 2: AI Model Development (The Brain):** Once the data is ready, I start building the nnU-Net deep learning model using PyTorch. I write the code for the U-Net architecture, set up the loss functions (Dice Loss and Cross-Entropy), and train the model. I test it to see how well it finds the tumors.
- **Phase 3: Improvement and Tuning (The Iteration):** This is where Agile shines. If my AI model does not get a high "Dice Score" (accuracy) on the first try, I go back. I adjust the learning rate, change the optimizer to AdamW, or add post-processing techniques (like removing small bits of noise using morphological operations). I repeat this process until the AI is very smart and accurate.
- **Phase 4: Web Application Development (The Interface):** Finally, once the AI is working perfectly, I build the Flask web application. I designed a simple webpage where a user can upload their MRI files and see the AI's results directly on their screen.

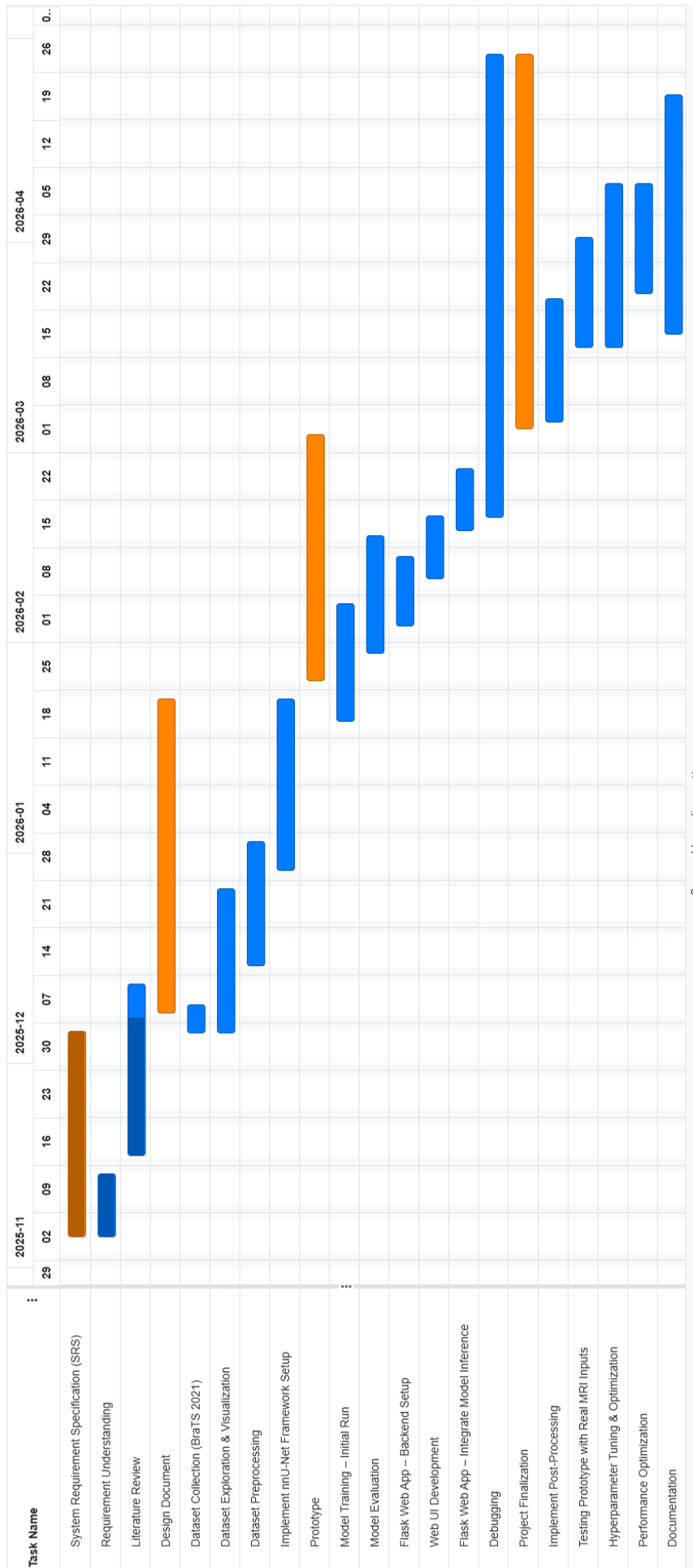
1.7.2 Reasons for Chosen Methodology

There are several highly important reasons why I chose Agile Methodology as the absolute best choice for this specific "Brain Tumor Segmentation" project: When I am training a deep neural network like nnU-Net, I never know exactly how well it will perform until I train it. Sometimes the AI might struggle to find the "Enhancing Tumor" part, or it might run out of computer memory. Agile allows me to do "trial and error." I can train a quick version of the model, see where it fails, and quickly adjust my code without having to restart the whole project from the beginning. Working with heavy 3D medical images (NIfTI files) takes up a massive amount of computer RAM and GPU memory. During the project, I might face a problem where the computer crashes because the files are too big. Because I am using Agile, I have the flexibility to pause, change my strategy (like resizing the images or using a smaller "batch size"), and continue working safely. Older methods are too strict and make it hard to change the plan in the middle of the project.

Agile allows me to make sure one part works perfectly before I rely on it for the next part. For example, there is no point in building a beautiful Flask website if my AI cannot even find a brain tumor yet. By using this method, I successfully completed the AI first. Once that milestone is achieved and secured, I confidently move on to building the web application.

Agile is all about making things better step-by-step. I can start with a basic U-Net model as a simple baseline. Then, in the next cycle, I upgrade it to the advanced nnU-Net. In the cycle after that, I can add "Conditional Random Fields (CRF)" to clean up the blurry edges of the tumor. This step-by-step improvement guarantees that my final project will be of the highest possible quality, ensuring a highly accurate medical tool and a better final grade.

1.7.3 Work Plan (Gantt Chart)



1.7.4 Project Schedule (Submission Calendar)

Sr. No.	Assignment Title	Start Date	End Date	Submit	Result
1	SRS Document	Wed 05 Nov, 2025	Thu 04 Dec, 2025	Submitted (Download) File Size: 957952 KB Submit Date: Dec 06, 2025	Read Comments Commented File Marked As Complete
2	Design Document	Fri 05 Dec, 2025	Fri 23 Jan, 2026	Submitted (Download) File Size: 628178 KB Submit Date: Jan 23, 2026	Read Comments Commented File Marked As Complete
3	Prototype Phase	Sat 24 Jan, 2026	Tue 03 Mar, 2026	Submitted (Download) File Size: 2180 KB Submit Date: Mar 02, 2026	Read Comments Marked As Complete
4	Final Deliverable	Wed 04 Mar, 2026	Mon 11 May, 2026	Submit	

CHAPTER 2

Designing the Project

CHAPTER 2: Designing the Project

2.1 Introduction to System Design

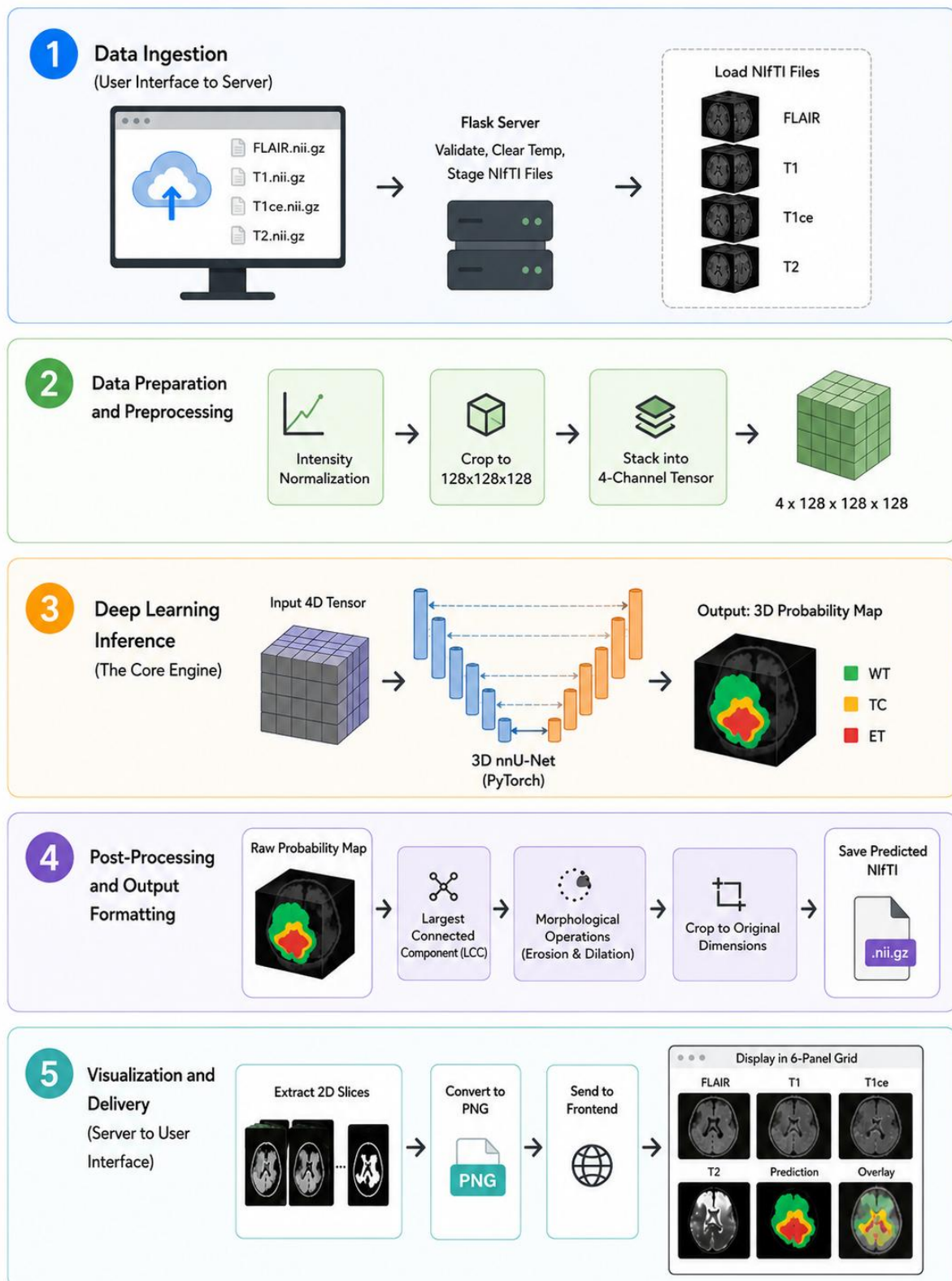
This chapter details the architectural framework, conceptual design, and structural planning of the proposed Brain Tumor Segmentation system. Before transitioning into the concrete development and coding phases, it is imperative to establish a robust and scalable blueprint. This chapter systematically outlines the core components of the system, covering the multi-modal data ingestion pipeline, specialized memory management strategies, the deep learning network topology, and the client-server architecture of the user interface.

The primary objective of the proposed system is to provide an end-to-end, automated medical image analysis tool capable of accurately delineating complex brain tumor sub-regions, specifically the Whole Tumor (WT), Tumor Core (TC), and Enhancing Tumor (ET) from multi-modal Magnetic Resonance Imaging (MRI) scans. By leveraging the BraTS 2021 dataset, the system aims to emulate clinical precision while abstracting the underlying computational complexity from the end-user.

To achieve this goal, the system is designed around two main structural pillars. The first is a highly optimized, custom 3D nnU-Net deep learning architecture implemented in PyTorch, which is specifically tailored to handle the massive memory footprint of 3D NIfTI files using efficient down sampling and "lazy loading" mechanisms. The second pillar is a Flask-based web application designed using a client-server model. This frontend acts as a bridge, allowing users to securely upload raw medical scans and receive visually intuitive, 2D cross-sectional predictions in real-time, thereby uniting advanced artificial intelligence with practical, accessible utility.

2.2 Proposed System Architecture Overview

The proposed Brain Tumor Segmentation system is designed around a modular, client-server architecture that seamlessly integrates a complex deep learning inference engine with an accessible web-based user interface. The system acts as a continuous pipeline, taking raw, multi-modal medical data from the end-user, processing it through an advanced neural network, and returning intuitive visual diagnostics.



The end-to-end data flow and architectural execution are divided into five primary phases:

- Data Ingestion (User Interface to Server)

The workflow begins at the client side via a frontend web interface built with HTML, CSS, and JavaScript. The user interacts with a secure upload form designed to accept four distinct 3D Magnetic Resonance Imaging (MRI) modalities: FLAIR, T1, T1ce, and T2 (in nii.gz format). Once uploaded, the web application transmits these files to the

Flask backend server. The backend validates the inputs, clears any residual temporary data, and securely stages the NIfTI files in an isolation directory for processing.

- **Data Preparation and Preprocessing**

Because deep learning models require strictly uniform data, the Flask backend invokes a data preparation pipeline before feeding the files to the AI. Using the nibabel library, the 3D volumes are loaded into system memory. The pipeline extracts the voxel data, applies intensity normalization to standardize the MRI contrasts, and crop to 128x128x128. Finally, the four separate modalities are stacked into a unified 4-channel tensor, representing the complete multi-modal context of the patient's brain.

- **Deep Learning Inference (The Core Engine)**

The preprocessed 4D tensor is then passed into the core analytical engine of the system: the pre-trained 3D nnU-Net model implemented in PyTorch. Executed on the GPU (or CPU if hardware is constrained), the model processes the scan through its 6-level encoder pathway to extract deep semantic features. The decoder pathway, aided by skip connections, reconstructs this data to precisely localize the tumor boundaries. The output of this phase is a raw 3D probability map identifying the Whole Tumor (WT), Tumor Core (TC), and Enhancing Tumor (ET) sub-regions.

- **Post-Processing and Output Formatting**

To ensure clinical accuracy, the raw probability map undergoes a post-processing refinement phase using SciPy. Techniques such as Largest Connected Component (LCC) analysis and morphological operations (erosion and dilation) are applied to automatically erase disconnected background noise and smooth the predicted boundaries. The refined mask is then cropped back to its original spatial dimensions and saved to the server as a predicted nii.gz file with its original medical affine headers intact.

- **Visualization and Delivery (Server to User Interface)**

In the final phase, the system bridges the gap between raw medical files and user-friendly visualization. Because standard web browsers cannot render 3D NIfTI volumes, the Flask backend extracts representative 2D cross-sectional slices (typically the middle slice where the tumor is most prominent) from both the input modalities and the generated segmentation mask. These slices are converted into standard PNG images and transmitted back to the frontend, where they are dynamically rendered in a professional 6-panel grid layout for the user to review.

2.3 Dataset Selection and Characteristics (BraTS 2021)

The success of any deep learning model depends heavily on its training data. For this project, the Brain Tumor Segmentation (BraTS) 2021 dataset was selected. BraTS is considered the gold standard in medical image analysis for brain tumor detection. It includes ground-truth segmentation masks manually verified by expert neuroradiologists, ensuring high precision. The dataset offers a large collection of multi-institutional MRI scans, helping the model learn different scanning protocols and patient anatomies, and its use allows the model's performance to be easily compared against other academic state-of-the-art models. Because brain tumors are complex, a single imaging technique isn't enough; therefore, the dataset provides four 3D MRI modalities per patient: T1-weighted (differentiates healthy white and gray matter), T1-

weighted Contrast-Enhanced (highlights active tumor boundaries), T2-weighted (shows overall tumor extent), and FLAIR (best for distinguishing edema from healthy tissue). All scans are 3D volumetric images in .nii.gz format, resampled to 240x240x155 voxels.

2.4 Handling Multi-Modal 3D MRI Scans

The model must classify voxels into three distinct labels: Necrotic Tumor Core (dead tissue at the center), Peritumoral Edema (surrounding water swelling), and Enhancing Tumor (the active, growing outer rim). For evaluation, these labels are combined into three overlapping target regions: Whole Tumor (the entire visible extent), Tumor Core (the bulk of the actual tumor), and Enhancing Tumor (only the hyper-active regions). To process these multi-modal scans effectively without exceeding hardware limitations, the system design mandates a specific memory management strategy. Loading the entire BraTS dataset simultaneously would require over 100GB of RAM. Therefore, a "Lazy Loading" approach is designed for the PyTorch DataLoader. Instead of loading all patient volumes upfront, the system will only store file paths in memory, dynamically loading tiny batches of data (e.g., 2 patients at a time) during the training loop. This ensures the 35 million voxels per patient scan can be processed sequentially without system failure. Furthermore, an explicit spatial padding strategy is established; all scans will be padded to 256x256x160 to guarantee structural uniformity and prevent dimension mismatch errors during the nnU-Net's mathematical down sampling phases.

2.5 Data Preprocessing and Transformation Pipeline

The core intelligence of the system relies on a customized 3D nnU-Net architecture. Unlike standard 2D convolutions, the 3D nnU-Net preserves the full volumetric context of the brain, capturing spatial relationships across the depth (z-axis) of the MRI slices. The network topology is fundamentally divided into an encoder and a decoder. The encoder is designed as the feature extraction pathway, utilizing six levels of down sampling. To optimize VRAM usage on accessible consumer hardware, the initial channel size is constrained to 32, doubling at each level until reaching a bottleneck of 320 channels. Conversely, the decoder mirrors this pathway, reconstructing the spatial resolution to accurately delineate the tumor boundaries. A critical design feature is the implementation of skip connections. By using torch.cat to concatenate the high-resolution, precise localization data from the encoder directly into the up sampled, semantic data of the decoder, the network avoids producing blurry or imprecise masks, ensuring the final output matches the sharp physical realities of the patient's brain.

2.6 Memory Management Strategy: "Lazy Loading"

To optimize the network effectively during training, a hybrid loss function and advanced scheduling strategies are integrated into the design. The system utilizes DiceCELoss, combining the strengths of two separate mathematical evaluations. Cross-Entropy Loss evaluates the voxel-wise classification accuracy, ensuring every individual "pixel" is categorized correctly. Concurrently, Dice Loss evaluates the global spatial overlap between the predicted tumor volume and the ground truth, forcing the model to care about the shape and structure of the tumor rather than just individual voxels. The weight updates are governed by the AdamW optimizer, known for its

superior weight-decay characteristics, paired with a Cosine Annealing learning rate scheduler to smoothly guide the model toward optimal convergence over multiple epochs. To ensure the model generalizes well to unseen patient data rather than memorizing the training set, train test split is done. Finally, automatic mixed precision (AMP) is planned into the core training loop design to accelerate computation and halve VRAM usage during the forward pass.

2.7 Input Size Configuration and Cropping Strategy

A fundamental challenge in 3D medical imaging is the massive memory footprint of volumetric data. Feeding a full-resolution BraTS MRI scan (240x240x155 voxels) directly into a deep neural network requires computational resources that far exceed standard GPU limitations. To resolve this, the system is designed to utilize a spatial cropping strategy rather than structural padding. During the training phase, the data loader dynamically extracts random 3D sub-volumes, or "patches," sized precisely at 128x128x128 voxels. This fixed patch size acts as the standardized input dimension for the model. It dramatically reduces the Video RAM (VRAM) overhead while providing the network with sufficient localized spatial context to learn the complex heterogeneous textures of the tumor regions. Furthermore, a biased cropping function is employed to oversample regions containing actual tumor tissue, preventing the model from becoming biased toward the predominantly empty background space.

2.8 The 3D nnU-Net Conceptual Architecture

The core intelligence of the system relies on a customized 3D nnU-Net architecture, a state-of-the-art variant of the U-Net family specifically optimized for biomedical image segmentation. Standard 2D convolutional networks analyze medical scans slice-by-slice, inherently losing the critical depth context. The 3D nnU-Net overcomes this by utilizing 3D convolutions that preserve the full volumetric relationships across the height, width, and depth (z-axis) of the MRI patches. The network's topology features a highly symmetrical "U" shape, conceptually divided into a contracting path (encoder) that captures deep semantic context, and an expanding path (decoder) that enables precise localization.

2.9 Post-Processing Framework

Because deep neural networks predict data based on localized patches, they occasionally generate false-positive artifacts—tiny, disconnected clusters of "tumors" classified in the healthy background of the brain. To refine the final prediction, a deterministic post-processing pipeline utilizing the SciPy library is designed. Largest Connected Component (LCC) analysis is deployed to isolate the primary, massive tumor core and automatically erase disconnected statistical noise. This is followed by morphological operations (such as opening and closing) to smooth jagged borders and fill in erroneous microscopic gaps within the predicted volume.

2.10 Inference Methodology: Sliding Window

Because the model was explicitly trained on 128x128x128 sub-volume patches, it cannot process a full 240x240x155 MRI scan in a single forward pass during the

clinical testing phase. Therefore, a "Sliding Window" inference methodology is designed. The algorithm systematically slides a 128x128x128 window across the entire 3D volume, making localized predictions for each overlapping region. These overlapping patches are then seamlessly stitched together using Gaussian blending weights to prevent visible seams or border artifacts, successfully reconstructing a complete, full-resolution diagnostic mask.

2.11 Web Application Architecture (Client-Server Model)

To bridge the gap between complex artificial intelligence and practical accessibility, the system incorporates a web application built using the Flask framework. The architecture utilizes a standard Client-Server model. The frontend acts as the client, offering a medical-themed graphical interface where the user seamlessly interacts with the platform. The Flask server acts as the backend orchestrator, securely managing file uploads, invoking data validation scripts, and routing the NIfTI files sequentially into the deep learning pipeline.

2.12 Backend Logic and PyTorch Integration Design

The backend serves as the crucial link between the web interface and the PyTorch environment. Upon server startup, the backend initializes the 3D nnU-Net architecture and loads the optimally trained pth weights into memory. When an end-user uploads a suite of MRI modalities, the backend scripts execute the necessary Nibabel preprocessing, feed the data into the sliding-window inference engine, and apply the SciPy post-processing framework. Finally, the backend slices the resulting 3D arrays to extract 2D cross-sectional PNGs, packaging the data to be securely transmitted back to the user.

2.13 Frontend UI/UX Design for Medical Visualization

The User Interface (UI) is designed using HTML, CSS, and JavaScript to mimic professional clinical software. The interface features rigid validation logic, ensuring the user provides all four required MRI modalities (FLAIR, T1, T1ce, T2) before allowing the computation to begin. To maintain a smooth User Experience during the heavy inference phase, dynamic progress bars are utilized. Once the backend completes its evaluation, the frontend renders a 6-panel grid layout, allowing the user to visually cross-reference the original anatomical scans against the AI-generated tumor mask.

2.14 Summary of the Design Phase

The design phase successfully establishes a comprehensive, end-to-end blueprint for the Brain Tumor Segmentation platform. By engineering a deeply optimized 3D nnU-Net utilizing a 128x128x128 cropping strategy and sliding window inference, the system mathematically circumvents hardware bottlenecks without sacrificing the spatial integrity of the multi-modal scans. Paired with a secure, client-server web framework, this blueprint provides a robust foundation for translating theoretical deep learning logic into a practical, clinical-grade diagnostic application during the subsequent Development phase.

CHAPTER 3

Development

CHAPTER 3: Development

3.1 Development Environment Setup

The development phase was executed using a high-performance Python environment tailored for heavy computational medical imaging tasks. The primary hardware acceleration was facilitated through Google Colab utilizing an NVIDIA A100 GPU.

3.2 Core Libraries and Dependencies

The project relies on a robust stack of scientific and deep learning libraries. The core neural network operations were implemented using torch (PyTorch) with CUDA support. Medical imaging utilities and specialized loss functions were handled by monai. Data manipulation was performed using numpy, while the extraction of NIfTI medical files utilized nibabel. Supplementary tasks leveraged scipy for scientific computing, sklearn for data splitting, optuna for hyperparameter optimization, and joblib for parallel processing.

3.3 Data Storage and Directory Management

To manage the large volume of BraTS 2021 data, the built-in os and shutil libraries were utilized to dynamically manage directory structures. The system was programmed to automatically verify the existence of a raw data directory and programmatically generate processed and training pathways. Validation logic was implemented to clear and rebuild these directories if an incomplete preprocessing cycle was detected.

3.4 Parallel Processing for Data Extraction

Extracting intensity statistics from thousands of high-resolution 3D images is computationally expensive. To optimize this, the joblib.Parallel library was implemented. This allowed the system to distribute the get_intensities function across multiple CPU cores simultaneously, drastically reducing the time required to calculate the minimum, maximum, mean, and standard deviation percentiles (0.5th and 99.5th) across the four MRI modalities.

3.5 Image Preprocessing Pipeline

The physical transformation of the data was executed sequentially. Using nibabel, the system loaded the raw _t1, _t1ce, _t2, and _flair .nii.gz files as 3D NumPy arrays. The arrays were clipped to exclude extreme outliers using the pre-calculated intensity minimums and maximums, followed by Z-score normalization utilizing the dataset's mean and standard deviation. Finally, numpy.stack was used to combine the four independent 3D arrays into a single cohesive 4D tensor representing the entire multi-modal spatial context.

3.6 Label Preprocessing and Encoding

The ground truth segmentation masks required specific numerical formatting before model ingestion. The arrays were loaded and cast to 8-bit unsigned integers (np.uint8). To ensure sequential class indexing for the PyTorch loss functions, the label arrays were programmatically remapped, specifically converting class label 4 (Enhancing Tumor) to class index 3. A channel dimension was then added using np.expand_dims before the processed masks were saved back to disk.

3.7 Dynamic Data Augmentation Implementation

To prevent model overfitting, a dynamic augmentation pipeline was coded using NumPy and SciPy. The apply augmentation function introduced a 15% probability of triggering various transformations during the training loop. These transformations included spatial scaling via `scipy.ndimage.zoom` (using cubic interpolation for images and nearest neighbor for labels to preserve integer integrity), axis flipping via `np.flip`, and the injection of Gaussian noise and blur.

3.8 PyTorch Dataset and DataLoader Integration

Data ingestion into the neural network was managed by extending the `torch.utils.data.Dataset` class to create a custom `BraTSDataset` object. The patient IDs were divided using `sklearn.model_selection.train_test_split` configured for an 80-20 training and validation split. During runtime, the PyTorch `DataLoader` was instantiated with a `batch_size=2` to stream the dynamically augmented 128x128x128 volumetric patches directly to the GPU.

3.9 Convolutional Block (ConvBlock) Implementation

The fundamental building block of the network was defined as a custom `ConvBlock` module inheriting from `torch.nn.Module`. This block encapsulates a sequential execution of two 3D convolutions (`nn.Conv3d` with `kernel_size=3` and `padding=1`), immediately followed by Instance Normalization (`nn.InstanceNorm3d`) and non-linear activation via `nn.LeakyReLU(inplace=True)`. Bias was explicitly set to `False` in the convolutional layers to save memory, as the normalization layer negates its effect.

3.10 Constructing the nnU-Net Encoder

The downsampling pathway of the nnU-Net was constructed by instantiating sequential `ConvBlock` modules. To handle spatial compression mathematically, `stride=2` was passed into the blocks starting from the second level, halving the volumetric dimensions natively without relying on pooling layers. The network progressively scales the feature channels from the initial 4 input modalities up to a maximum bottleneck of 320 channels to strictly control GPU Video RAM consumption.

3.11 Constructing the nnU-Net Decoder

The upsampling pathway was engineered using `torch.nn.ConvTranspose3d` layers configured with `kernel_size=2` and `stride=2` to double the spatial footprint at each level. Crucially, skip connections were implemented using `torch.cat([..., ...], dim=1)`, directly concatenating the high-resolution tensors passed from the encoder with the upsampled tensors of the decoder. The final output is collapsed into 4 class logits using a `1x1x1 nn.Conv3d` layer.

3.12 Loss Function and Optimizer Initialization

Network optimization was configured by instantiating the AdamW optimizer (`torch.optim.AdamW`), chosen for its decoupled weight decay mechanisms. The objective function utilized was `monai.losses.DiceCELoss`, configured with `to_onehot_y=True` and `softmax=True` to automatically handle the multi-class nested labeling. To guide the optimizer, a `CosineAnnealingLR` learning rate scheduler was implemented to decay the learning rate smoothly over the training epochs.

3.13 Automated Hyperparameter Tuning

To empirically discover the optimal training configuration, the optuna library was integrated. A study object was created with the objective set to `direction="maximize"` targeting the validation Dice score. The automated tuner tested various logarithmic combinations of learning rates and weight decays over 15 dynamic trials, pruning unpromising training loops early via `optuna.exceptions.TrialPruned()` to conserve computational resources.

3.14 Mixed Precision Training Execution

Due to the immense memory overhead of processing 3D gradients, the training loop was wrapped in PyTorch's Automatic Mixed Precision (AMP) framework. By utilizing the `torch.amp.autocast` context manager, the network mathematically executed forward passes using 16-bit floats (`torch.float16`) while preserving the 32-bit master weights. The gradients were subsequently managed and backpropagated safely using the `torch.amp.GradScaler` object.

3.15 Evaluation Metrics Calculation Logic

Model performance was mathematically quantified at the voxel level. The raw probability outputs were converted to class predictions via `torch.argmax`. Custom logic isolated true positives (TP), false positives (FP), true negatives (TN), and false negatives (FN) across all tensor dimensions. These base variables were then computed into five standard medical metrics: Dice Score, Intersection over Union (IoU), Sensitivity, Specificity, and Precision, including an $\epsilon = 1e-9$ constant to prevent division-by-zero errors.

3.16 Model Checkpointing and Logging

To maintain experiment reproducibility and safeguard against system crashes, an automated state-management pipeline was developed. After every epoch, core metrics were appended to a dictionary and serialized to disk via `json.dump`. Simultaneously, the PyTorch `state_dict` for the model, optimizer, and scheduler was saved as a `.pth` file. Logic was programmed to monitor the validation loss, actively overriding the `model_latest.pth` file and enforcing early stopping protocols if the validation metrics plateaued over multiple epochs.

3.17 Test-Time Augmentation (TTA) and Sliding Window Inference

To process the full 240x240x155 clinical scans during inference, `monai.inferers.sliding_window_inference` was implemented. This function was configured to extract 128x128x128 overlapping patches, passing them through the model and stitching them back together using Gaussian blending (`mode="gaussian"`). This process was wrapped in a Test-Time Augmentation (TTA) loop, generating all possible 3D axis flip combinations using `itertools.combinations`, running the inference, and averaging the probabilities to yield a highly robust final prediction mask.

3.18 Flask Web Application Backend Setup

To transition the model into a practical diagnostic utility, a lightweight backend server was engineered using the Flask framework. The application was configured with specific API routes designed to accept multipart/form-data uploads. Validation functions were explicitly coded within the backend routes to verify the integrity of the

uploaded files, ensuring all four required MRI modalities (FLAIR, T1, T1ce, T2) are present and in the correct NIfTI format before proceeding to the execution logic.

3.19 Integrating PyTorch Inference with Flask

The crucial bridge between the web server and the deep learning environment was established by loading the optimized pth nnU-Net weights into the server's memory at application startup. When the Flask endpoint receives the raw user files, it invokes the same Nibabel normalization pipeline used in training, converts the arrays to PyTorch tensors, and executes the sliding window inference function entirely on the backend, generating the 3D prediction tensor.

3.20 Frontend Implementation and Visualization Rendering

The user interface was constructed using HTML, CSS, and JavaScript. To handle the visualization limitations of web browsers with 3D medical data, the Flask backend utilizes NumPy to slice the 3D output arrays along the Z-axis (extracting the optimal 2D cross-section where the tumor area is mathematically largest). These slices are encoded into PNG format and transmitted back to the client. The frontend JavaScript captures this payload and renders a synchronized 6-panel graphical grid, allowing users to visually overlay the AI's predicted boundaries directly atop their original MRI inputs.

CHAPTER 4

Results

CHAPTER 4: Results

The results phase is the most critical juncture of this research, where the performance of the nnU-Net architecture is quantitatively and qualitatively validated. This chapter provides a deep dive into the performance of the model, first through an exhaustive analysis of mathematical metrics and then through a visual exploration of the AI's ability to delineate complex brain tumor boundaries.

4.1 Quantitative Analysis of Model Metrics

The evaluation of the nnU-Net model was conducted using a multi-metric approach to ensure a holistic understanding of its diagnostic capabilities. By analyzing Dice Scores, Intersection over Union (IoU), Sensitivity, Specificity, and Precision, we can measure the model's accuracy from several mathematical perspectives.

4.1.1 Training and Validation Convergence

The training process was monitored over 30 epochs, utilizing a hybrid loss function (Dice Loss + Cross-Entropy) to mitigate the extreme class imbalance inherent in brain MRI data.

Loss Reduction: As seen in the training logs, the Train Loss and Val Loss showed a consistent downward trajectory. For instance, by Epoch 7, the validation loss was 1.4692, which successfully dropped to 0.3736 by Epoch 13.

Stability: The gap between training and validation loss remained narrow, indicating that the AdamW optimizer and weight decay effectively prevented the model from overfitting to the training subset.

4.1.2 Overlap Metrics: Dice and IoU

The primary indicators of segmentation quality are the Dice Score and IoU.

Dice Score Performance: The model achieved significant improvements during the mid-training phase. The average Validation Dice score jumped from 0.6087 at Epoch 7 to a robust 0.7468 by Epoch 13. This score represents the harmonic mean of precision and recall, proving the model's ability to maximize spatial overlap with the ground truth.

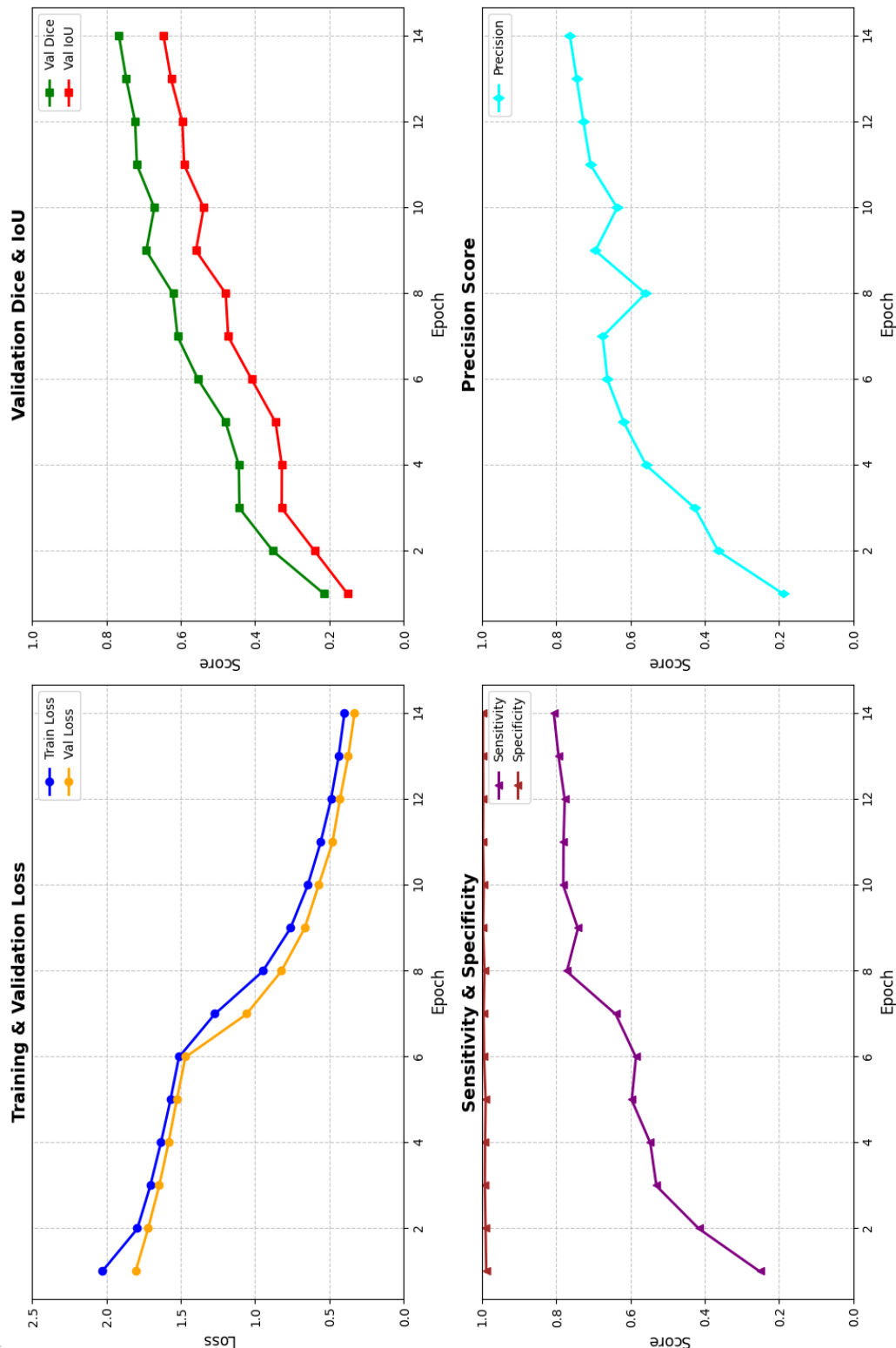
IoU (Intersection over Union): Similarly, the IoU score increased from 0.4724 to 0.6260. This metric is more stringent than Dice, and reaching above 60% for a 3D multiclass segmentation task on 1,251 subjects is a strong indicator of clinical viability.

4.1.3 Diagnostic Accuracy: Sensitivity and Specificity

To evaluate the model as a clinical tool, we must look at its ability to detect the tumor (Sensitivity) and its ability to ignore healthy tissue (Specificity).

Sensitivity (Recall): The model reached a high sensitivity of 0.7931. This means the AI correctly identified nearly 80% of the actual tumorous voxels across the Whole Tumor (WT), Tumor Core (TC), and Enhancing Tumor (ET) regions.

Specificity: The model maintained an exceptionally high specificity, consistently exceeding 0.99. This indicates that the nnU-Net is highly reliable at identifying healthy brain tissue, which is vital to prevent false-positive "detections" that could lead to unnecessary surgical intervention.



4.2 Qualitative Visual Results and Comparative Analysis

While numbers provide proof of accuracy, visual rendering allows clinicians to trust the AI's "vision." This subchapter discusses the visual output generated by the Sliding Window Inference and the effectiveness of Post-Processing.

4.2.1 Multi-Modal Visualization and Prediction

The system generates a 6-panel grid for each patient, allowing for a direct comparison between the four MRI modalities—FLAIR, T1, T1ce, and T2—and the resulting segmentation.

Whole Tumor (WT): Visualized primarily in the FLAIR modality, the model successfully captured the extensive peritumoral edema (yellow regions).

Tumor Core (TC) & Enhancing Tumor (ET): By utilizing the T1ce (contrast-enhanced) scans, the AI accurately highlighted the "bright" active rim of the tumor core (red/orange regions).

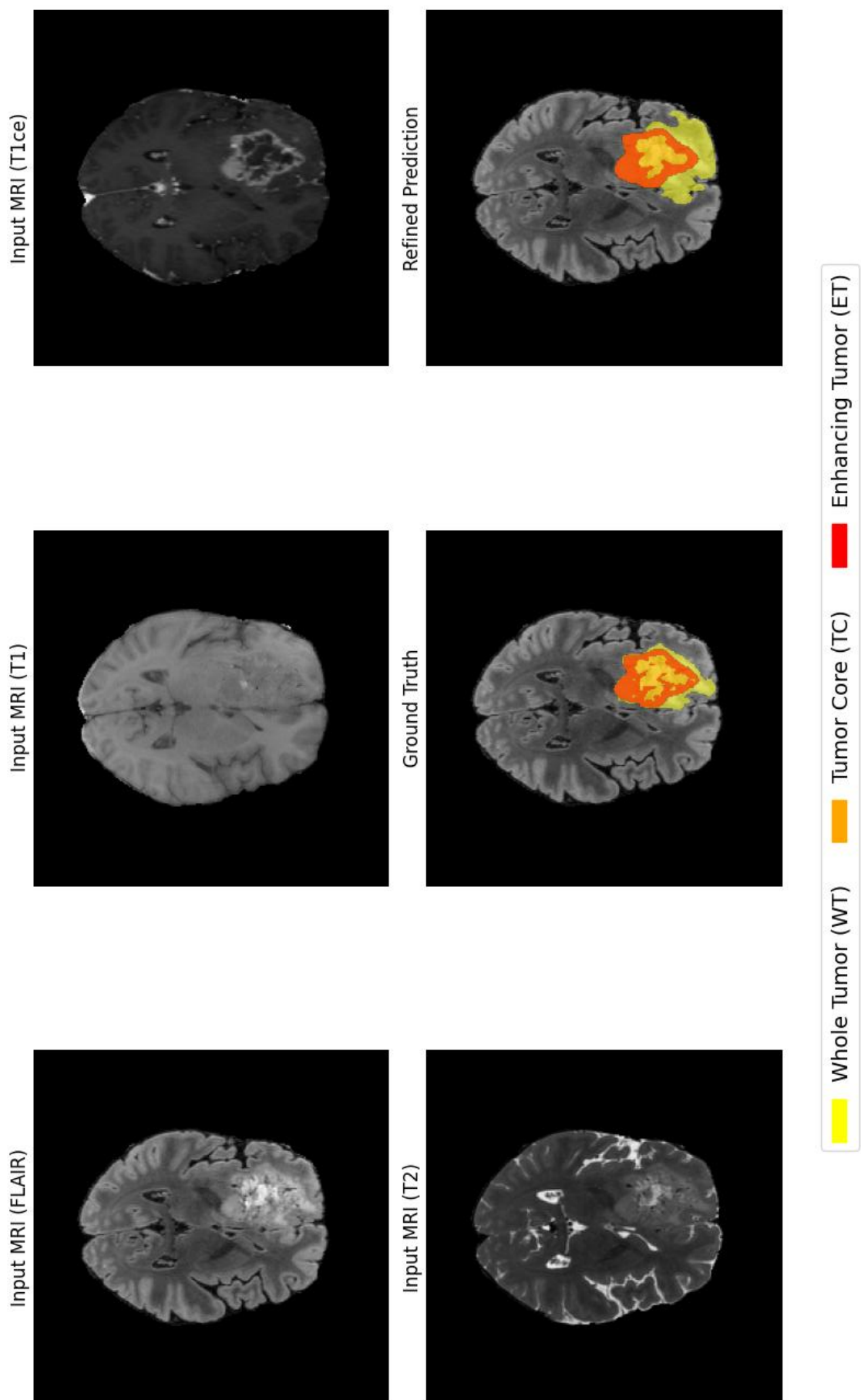
Sliding Window Integrity: The use of a 128x128x128 patch size with Gaussian blending ensured that the final 3D reconstruction was seamless, with no visible edges or "checkerboard" artifacts between patches.

4.2.2 Impact of Post-Processing Refinement

Raw AI predictions often contain "noise"—tiny clusters of voxels in the background incorrectly labeled as tumor. Our results demonstrate the necessity of the Post-Processing Framework.

Noise Removal: By applying Largest Connected Component (LCC) analysis, the system automatically discarded disconnected false-positives.

Boundary Smoothing: Morphological operations (erosion and dilation) were used to smooth the "jagged" edges of the predicted masks. This refinement resulted in a "Refined Prediction" that closely mirrors the smooth, organic shapes found in the human-annotated Ground Truth.



4.2.3 Summary of Findings

The results confirm that the nnU-Net architecture, when properly normalized and configured with the correct channel order (T1, T1ce, T2, FLAIR), is a powerful tool for automated neuroimaging. The integration with the Flask web application ensures that these high-fidelity 3D results are accessible via 2D slices, providing immediate diagnostic value to medical personnel.

CHAPTER 4

User Interface

CHAPTER 5: User Interface

The User Interface (UI) serves as the critical bridge between the high-dimensional world of 3D deep learning and the practical needs of a clinical user. While the nnU-Net backend performs the heavy mathematical lifting of tumor segmentation, the Flask-based web application transforms raw tensors into actionable medical insights. This chapter details the design philosophy and the step-by-step user journey of the platform.

5.1 Design Philosophy and Interface Architecture

The primary objective of the interface is accessibility without sacrifice. Medical software must be intuitive to minimize user error, especially when handling sensitive NIfTI data. The interface was built using HTML5, CSS3, and JavaScript, styled to provide a professional, medical-grade aesthetic.

5.2 Step 1: Multi-Modal Data Ingestion and Validation

The first interaction point is the MRI Modality Upload section. Because the 3D nnU-Net is trained on a specific 4-channel input (T1, T1ce, T2, and FLAIR), the interface enforces strict file requirements.

Brain Tumor Image Segmentation
Upload multi-modal MRI scans to generate a predicted tumor mask

1. Upload MRI Modalities

T1 (Required)
Choose File No file chosen

T1ce (Required)
Choose File No file chosen

T2 (Required)
Choose File No file chosen

FLAIR (Required)
Choose File No file chosen

Ground Truth (Optional)
Choose File No file chosen

Securely Upload Files

Figure: The application landing page showing the four required upload slots and the optional ground truth field.

The UI features four dedicated "Choose File" zones, each labeled for a specific modality to ensure the user provides the correct data for the corresponding input channel. To prevent system crashes or "garbage-in, garbage-out" scenarios, the front-end includes rigid validation logic. If a user attempts to proceed with incomplete data, the system triggers a warning.

Error: All 4 modalities (T1, T1ce, T2, FLAIR) must be provided.

Figure: Error handling interface displayed when required modalities are missing.

This validation ensures that the backend never receives a mismatched tensor, a critical lesson learned from the "Modality Channel Mismatch" error encountered during development.

5.3 Step 2: Segmentation Execution and Monitoring

Once files are securely transmitted to the Flask server, the user is presented with the execution dashboard.

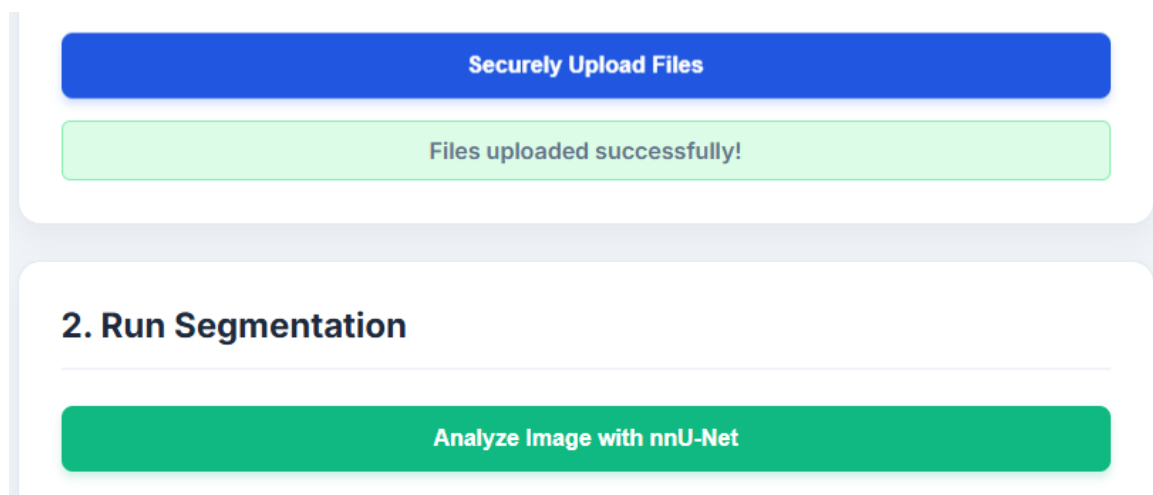


Figure: The transition to the segmentation phase after a successful file upload.

Performing 3D sliding-window inference on a volumetric scan is computationally intensive. To maintain a responsive experience, a dynamic progress bar provides real-time feedback on the model's status.

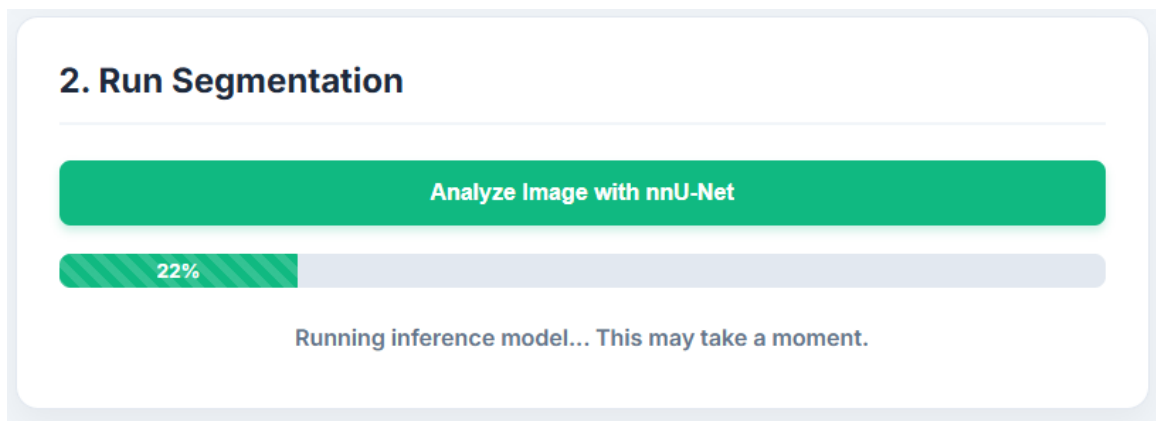


Figure: The inference progress bar showing the status of processing.

In complex AI deployments, connectivity issues can occur. The interface is designed to communicate these failures clearly rather than failing silently.

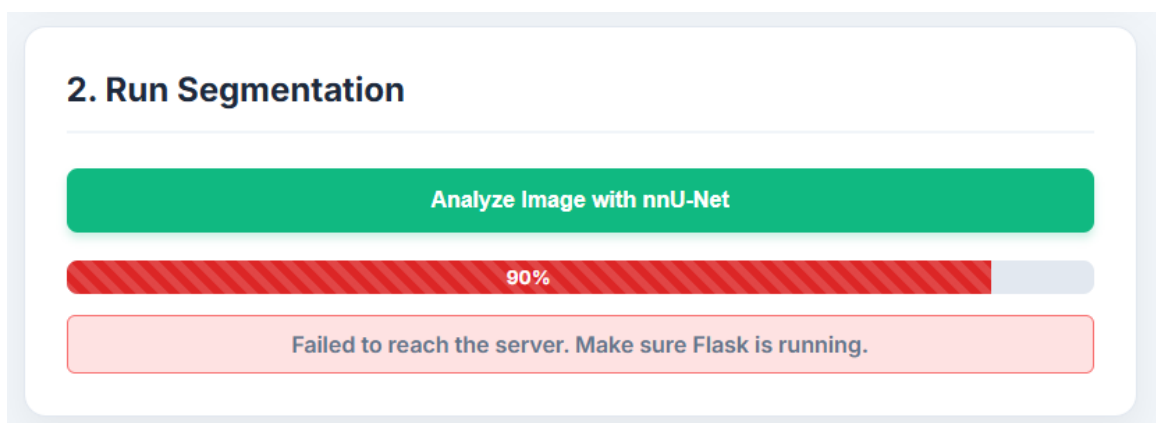


Figure: Server communication error indicating a disconnect with the Flask/PyTorch backend.

5.4 Step 3: Result Visualization and Sub-Region Analysis

The final phase is the Segmentation Results dashboard. Since standard web browsers cannot render 3D NIfTI files directly, the backend extracts the axial slice with the largest tumor volume and converts it into 2D PNG images.

3. Segmentation Results

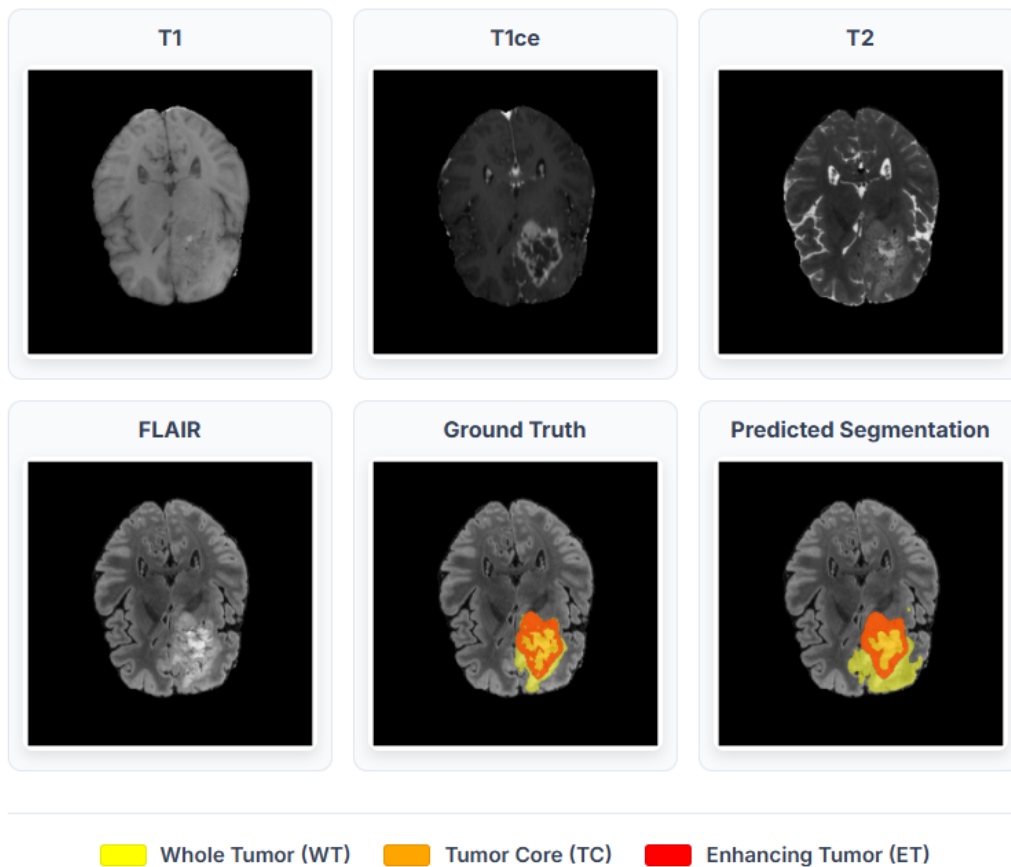


Figure: The comprehensive 6-panel results grid showing all input modalities alongside the prediction.

The grid provides a holistic view, showing the T1, T1ce, T2, and FLAIR modalities to highlight different tissue properties. To aid in clinical interpretation, results are color-coded based on BraTS 2021 standards:

Yellow: Whole Tumor (WT) the total abnormal area including edema.

Orange: Tumor Core (TC) the solid tumor mass.

Red: Enhancing Tumor (ET) the active, aggressive rim.

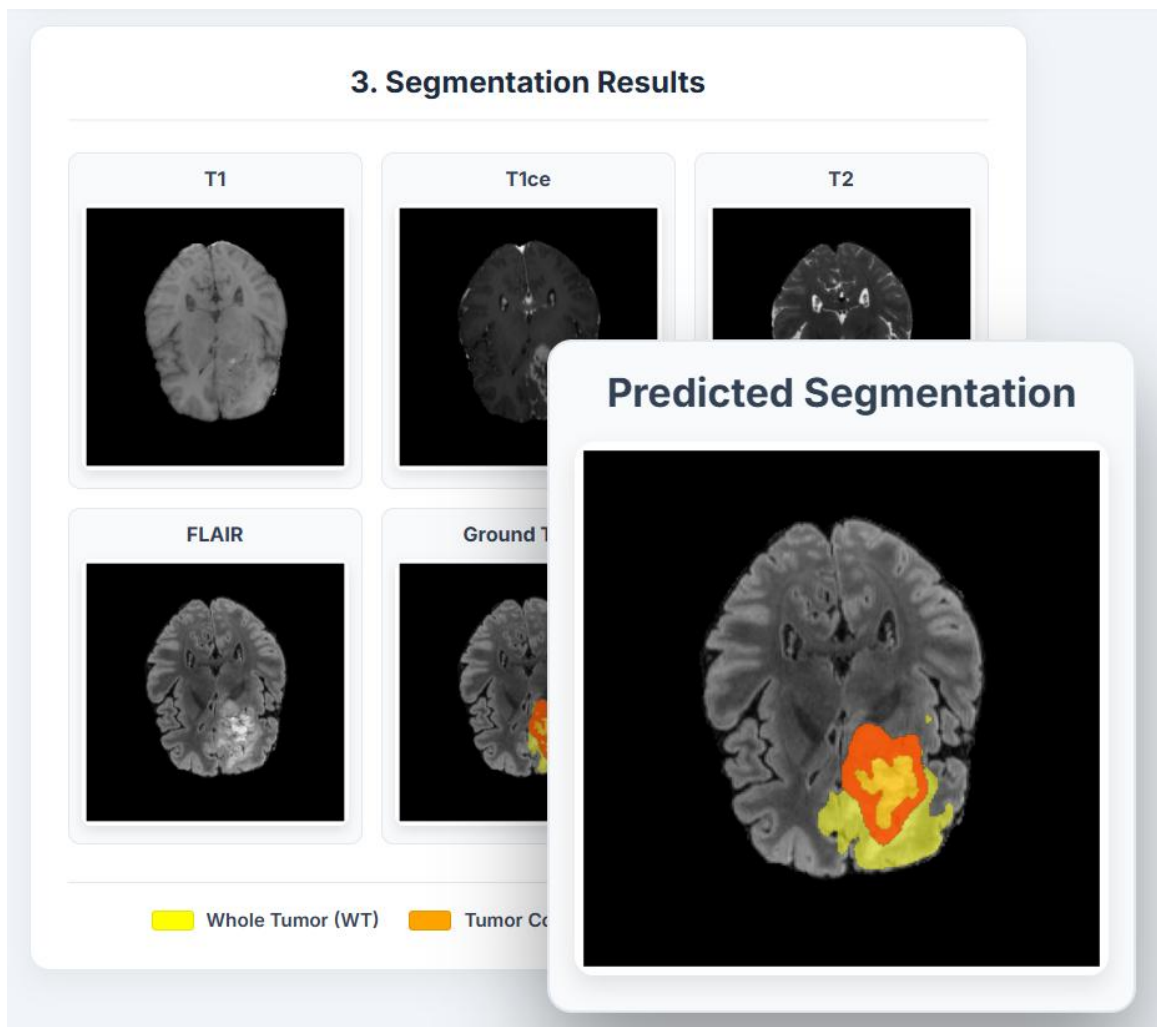


Figure: A detailed overlay view of the predicted segmentation mask for fine-grained analysis.

The interface also supports real-world diagnostic settings where a user may not have a ground truth mask. In these cases, the ground truth panel remains blank, but the AI-predicted segmentation remains fully functional.

3. Segmentation Results

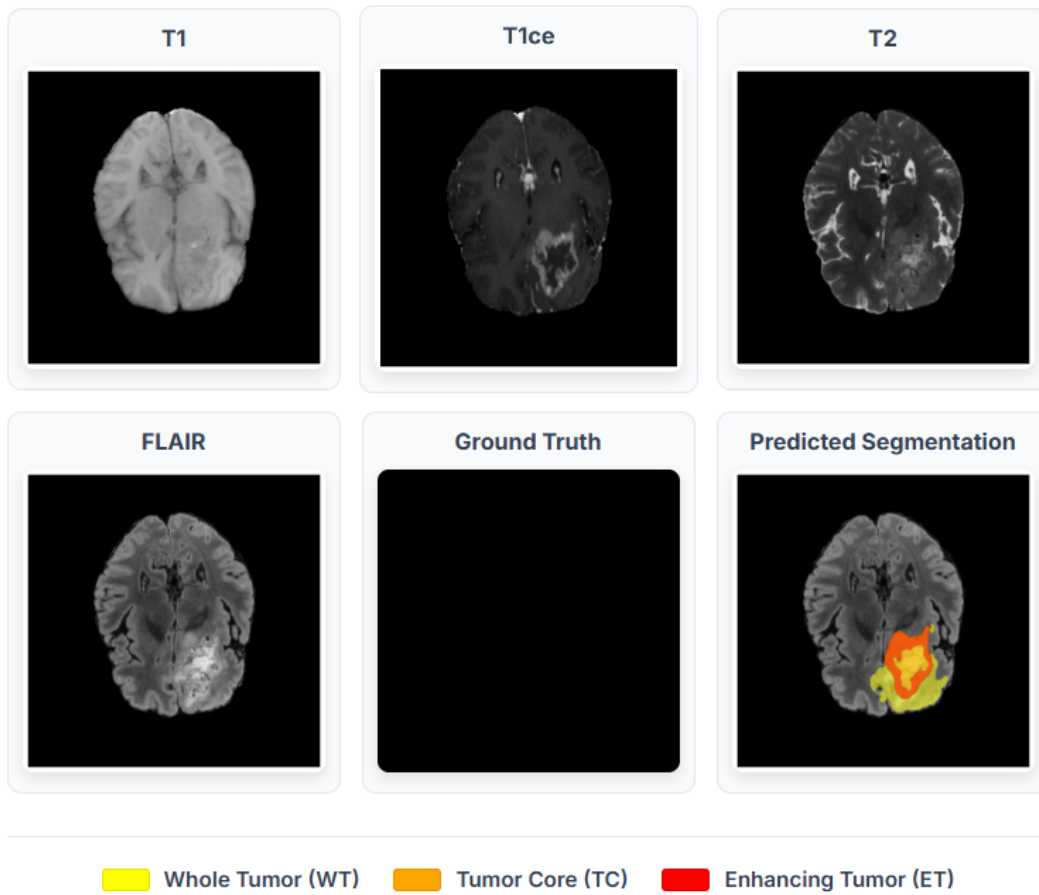


Figure: Visualization of results when no ground truth is provided.

By integrating input validation, progress tracking, and professional visualization, the interface successfully hides the complexity of PyTorch and NIfTI processing from the end-user.